

3 weird tricks to optimize your MMC

How to optimize non-differentiable objectives

Motivation

- Scoring functions on Numerai have been non-differentiable for a long time

Motivation

- Scoring functions on Numerai have been non-differentiable for a long time
 - The original corr was spearman correlation.

Motivation

- Scoring functions on Numerai have been non-differentiable for a long time
 - The original corr was spearman correlation.
 - Non differentiable due to ranking (although can be relaxed).

Motivation

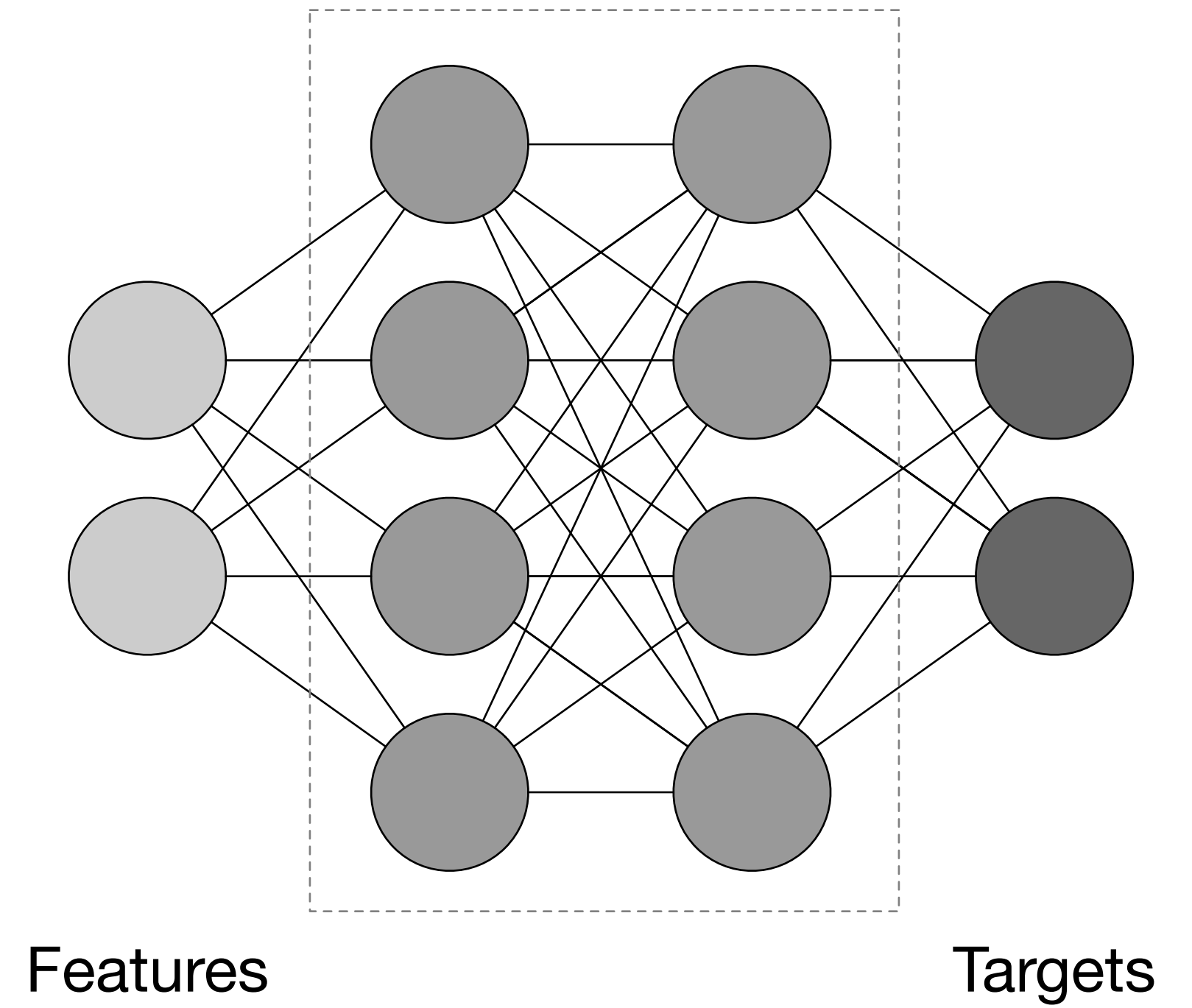
- Scoring functions on Numerai have been non-differentiable for a long time
 - The original corr was spearman correlation.
 - Non differentiable due to ranking (although can be relaxed).
- TC was blackbox and we were told to optimize for FNCv3.

Motivation

- Scoring functions on Numerai have been non-differentiable for a long time
 - The original corr was spearman correlation.
 - Non differentiable due to ranking (although can be relaxed).
 - TC was blackbox and we were told to optimize for FNCv3.
 - Numerai corr and MMC are also the same.

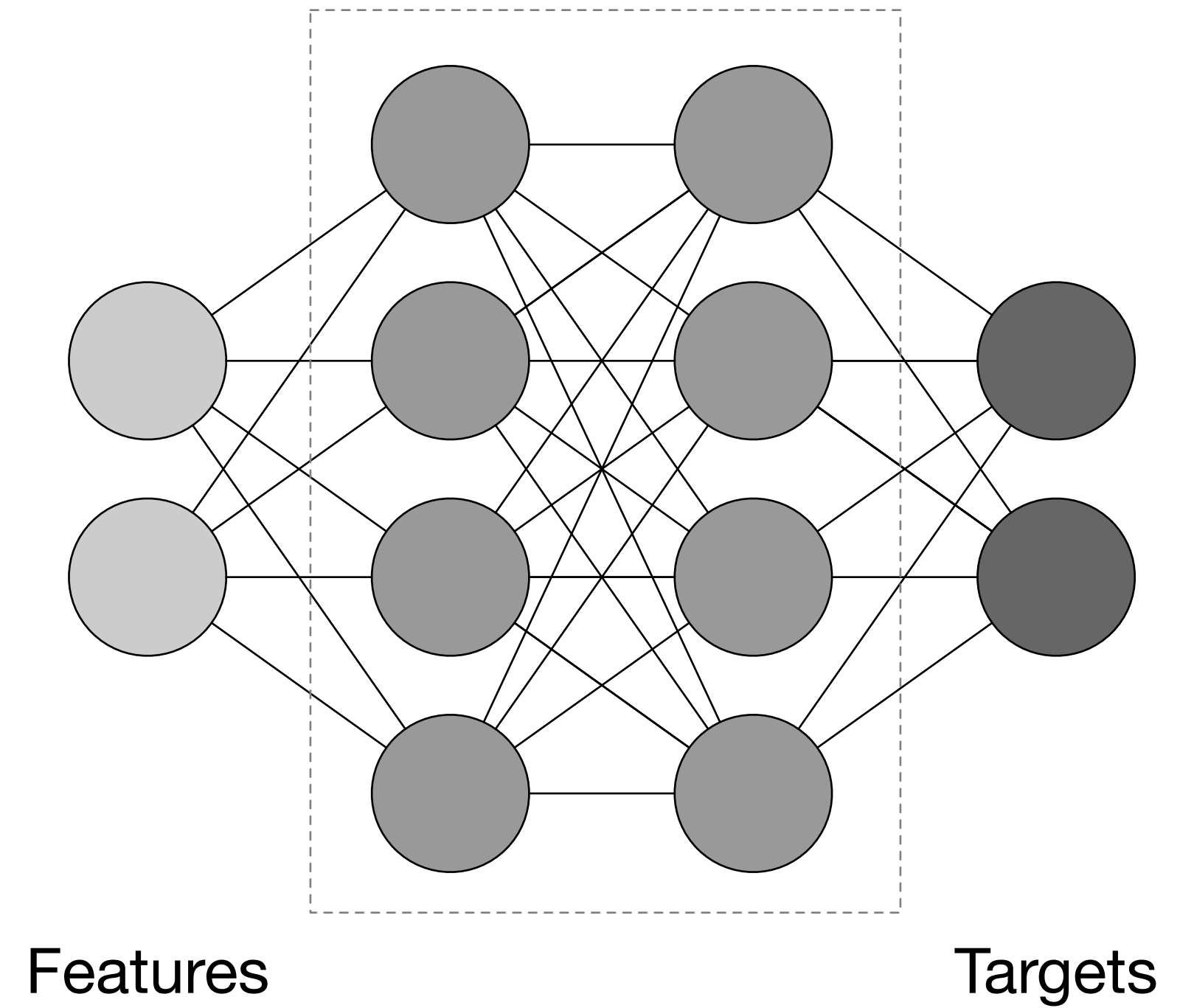
Neural Baseline

- Multi task NN



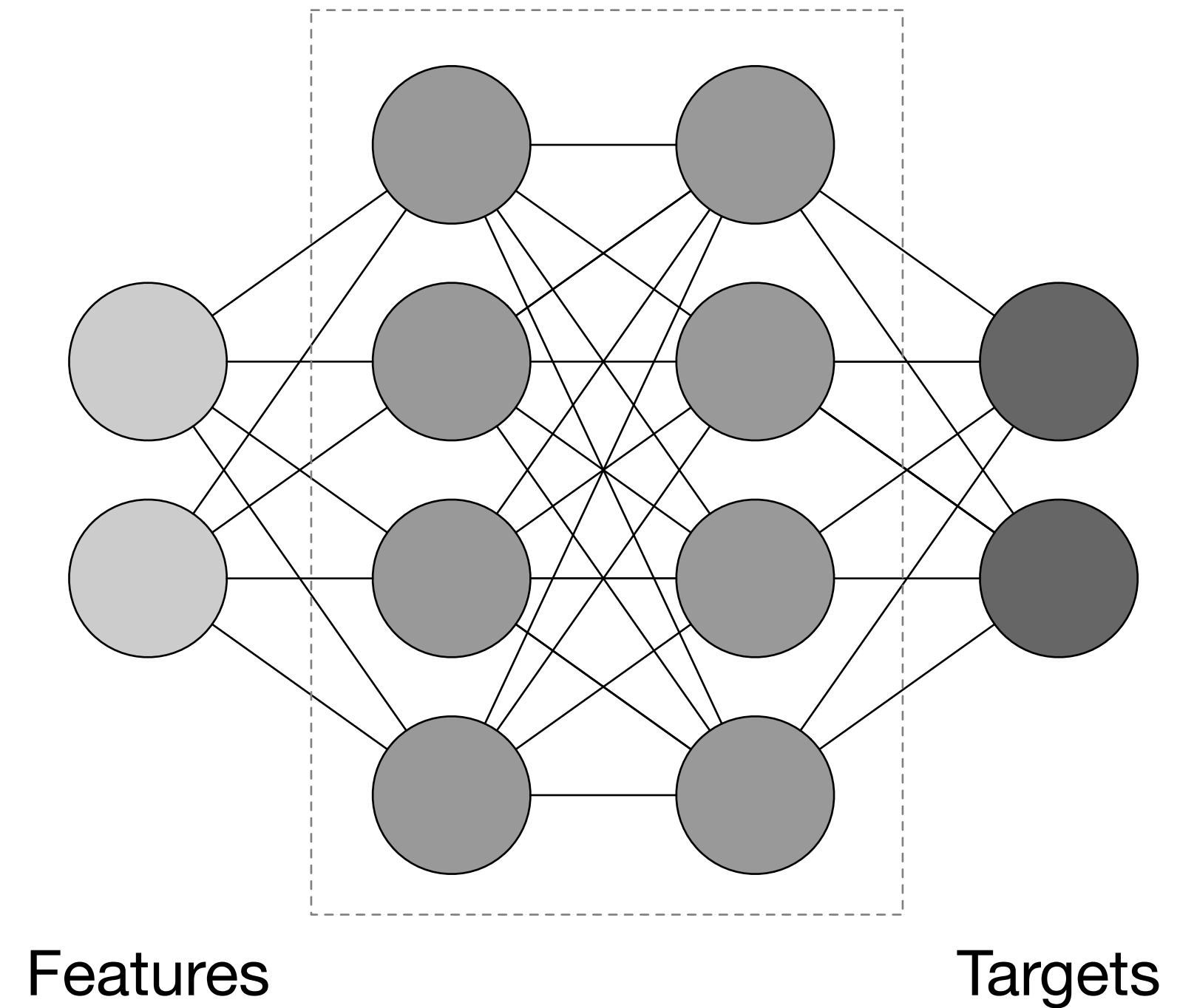
Neural Baseline

- Multi task NN
- Pre-trained on all targets with a simple objective



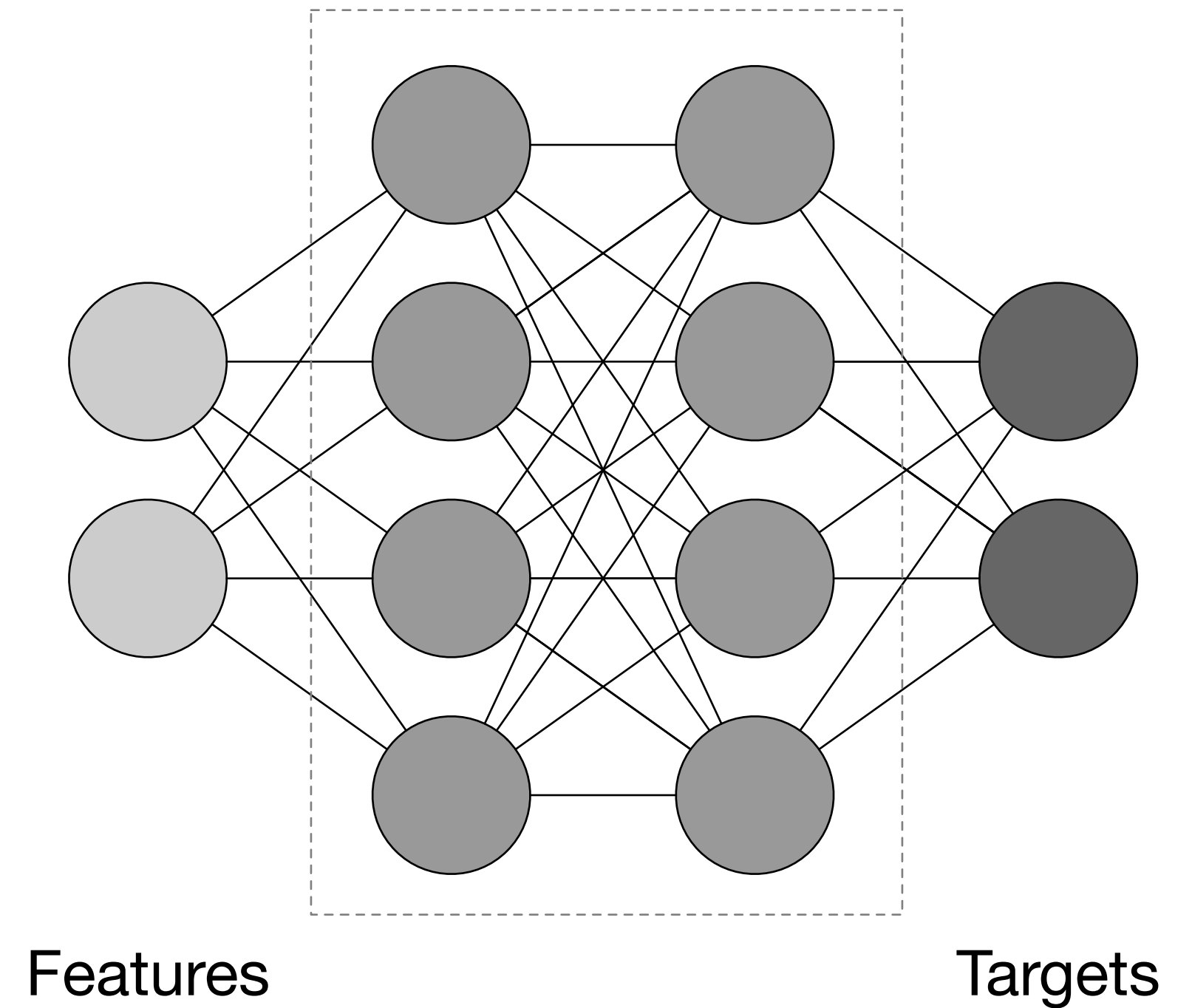
Neural Baseline

- Multi task NN
- Pre-trained on all targets with a simple objective
- Later fine tuned on a single target



Neural Baseline

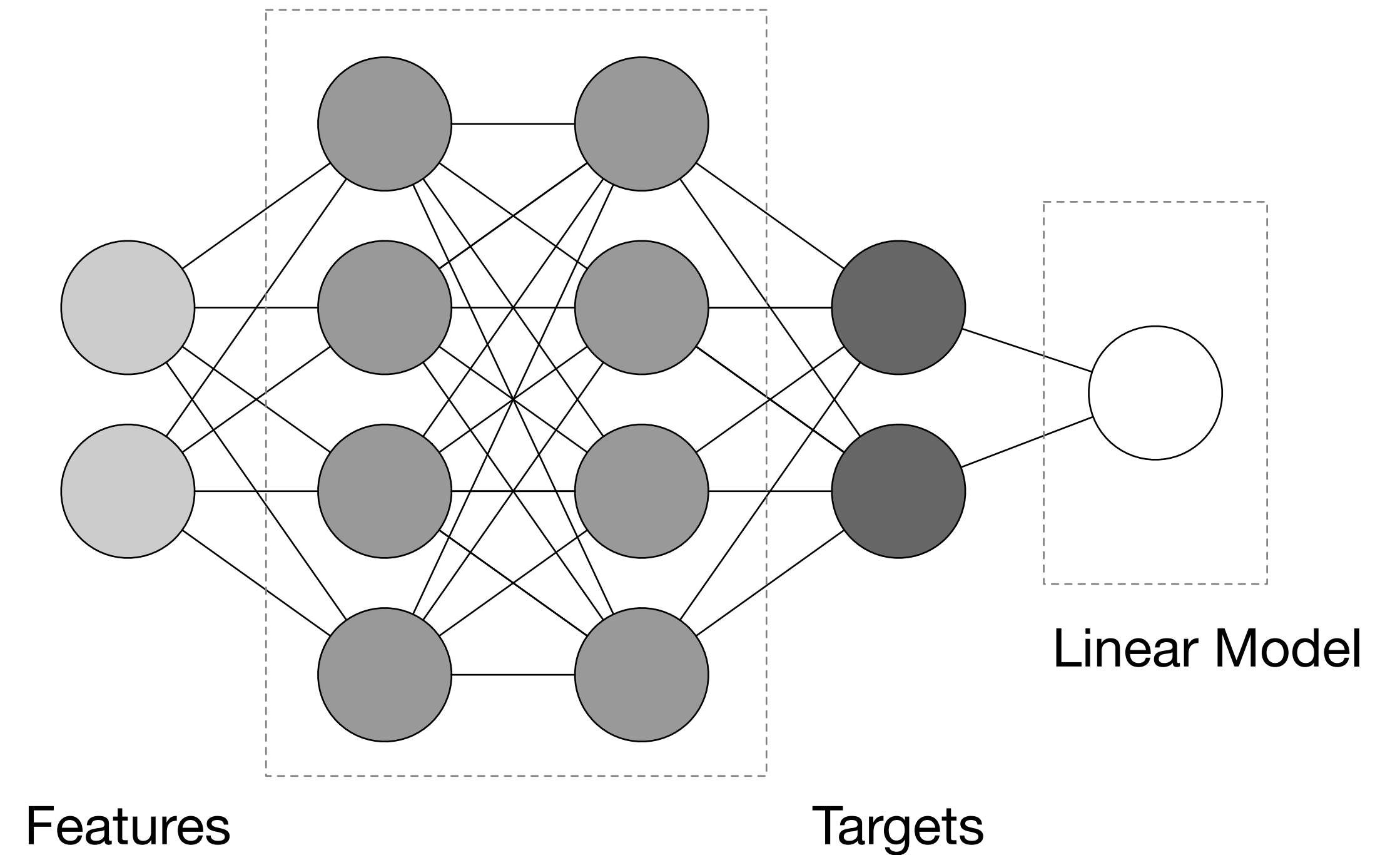
- Multi task NN
- Pre-trained on all targets with a simple objective.
- Later fine tuned on a single target.
- As it turns out, the pre-trained network is almost always performs better than the single task fine tuned network on live data.



Trick #1

Evolutionary optimization

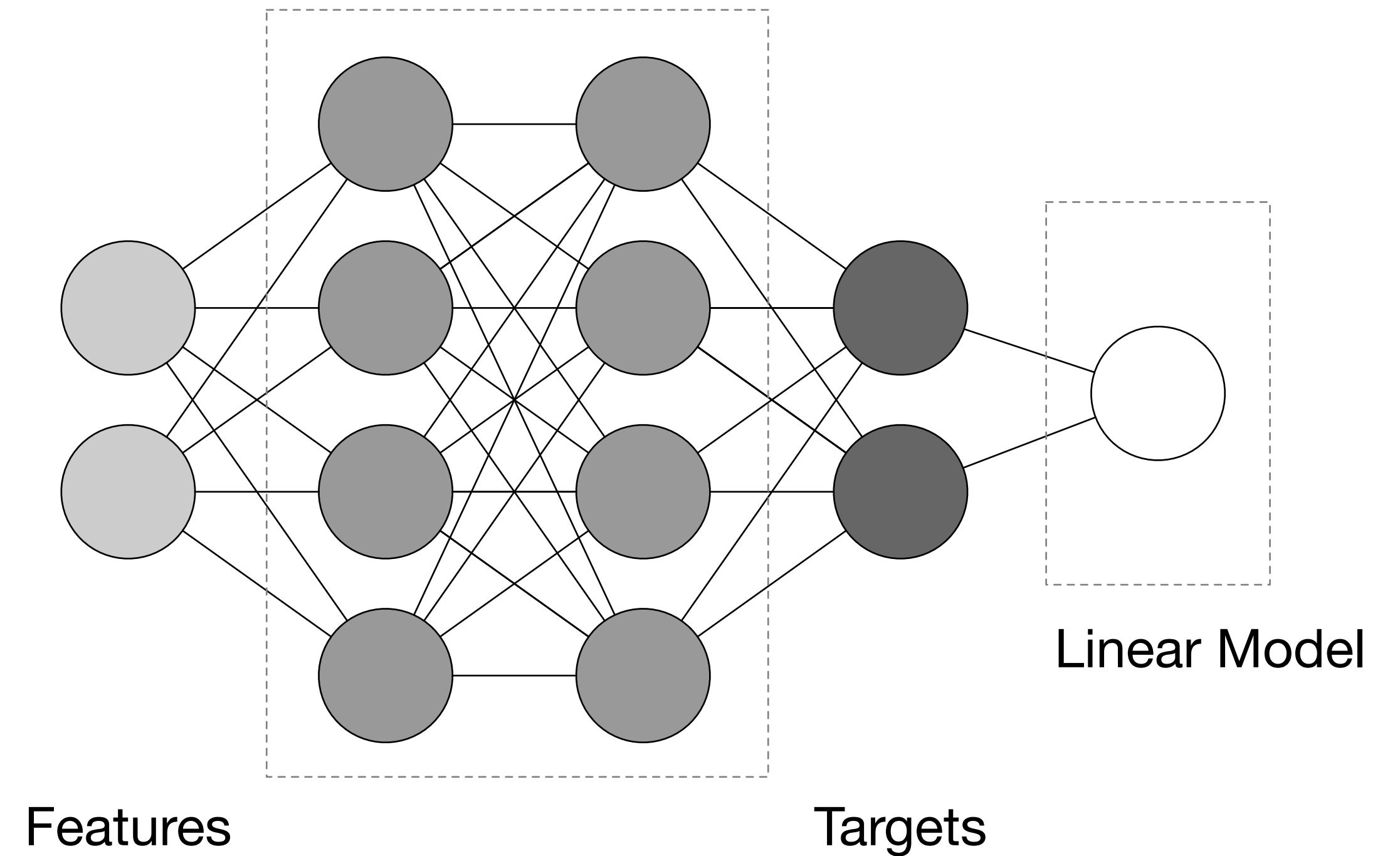
- Take the pre-trained multitask NN and train a linear model



Trick #1

Evolutionary optimization

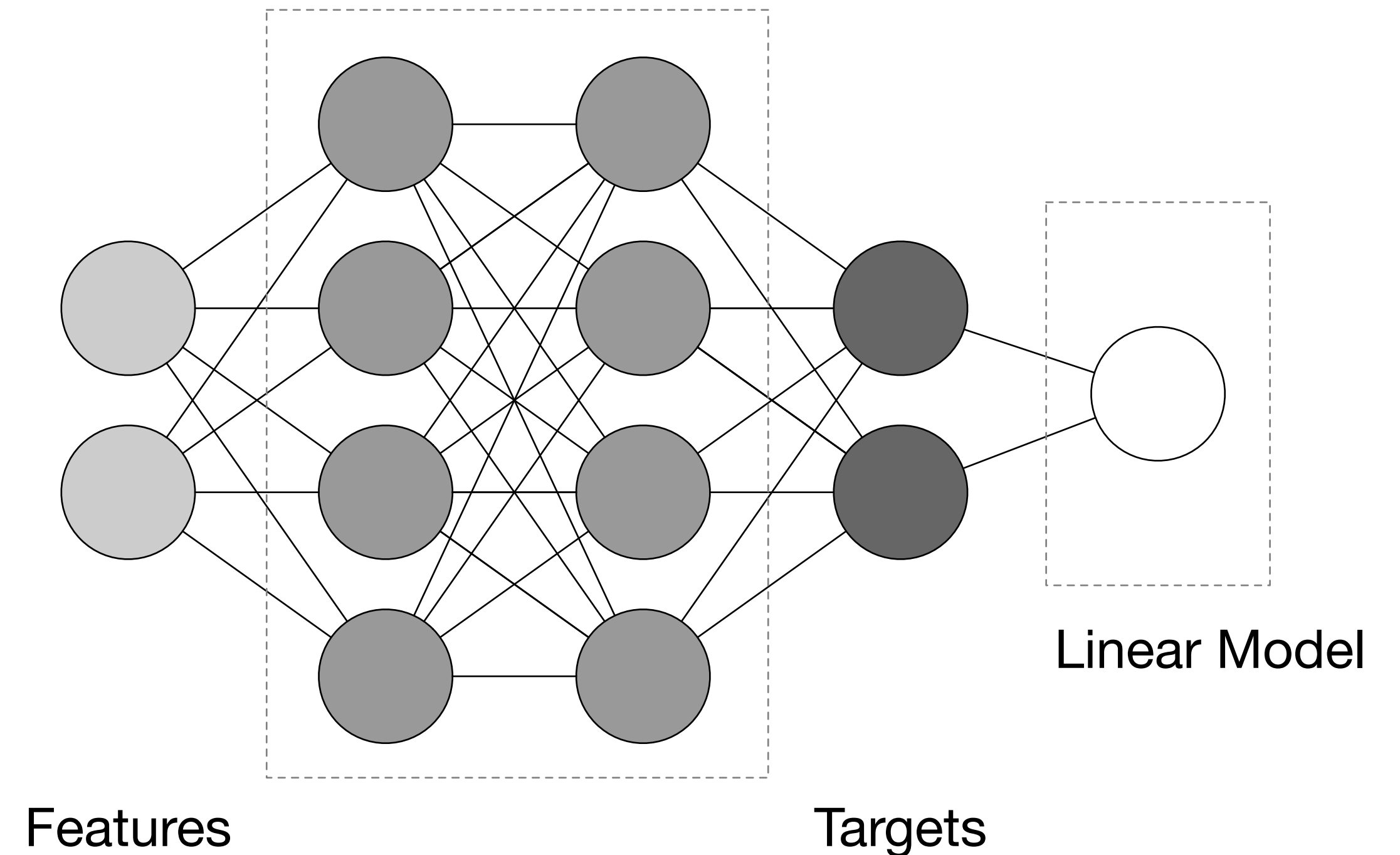
- Take the pre-trained multitask NN and train a linear model
- Weights of the Linear model are optimized by a gradient-free optimizer



Trick #1

Evolutionary optimization

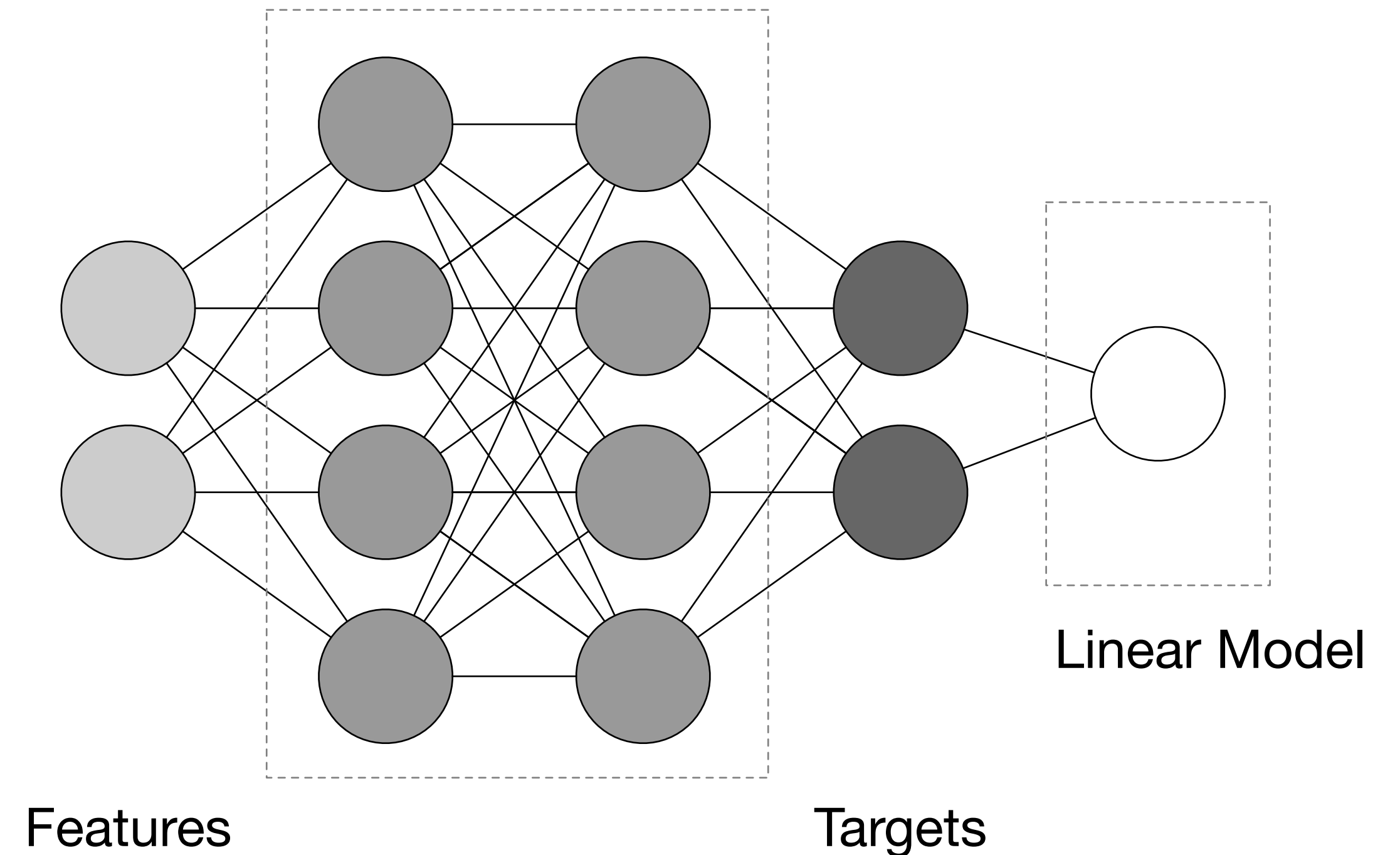
- Take the pre-trained multitask NN and train a linear model
- Weights of the Linear model are optimized by a gradient-free optimizer
- Tried a few different optimizers



Trick #1

Evolutionary optimization

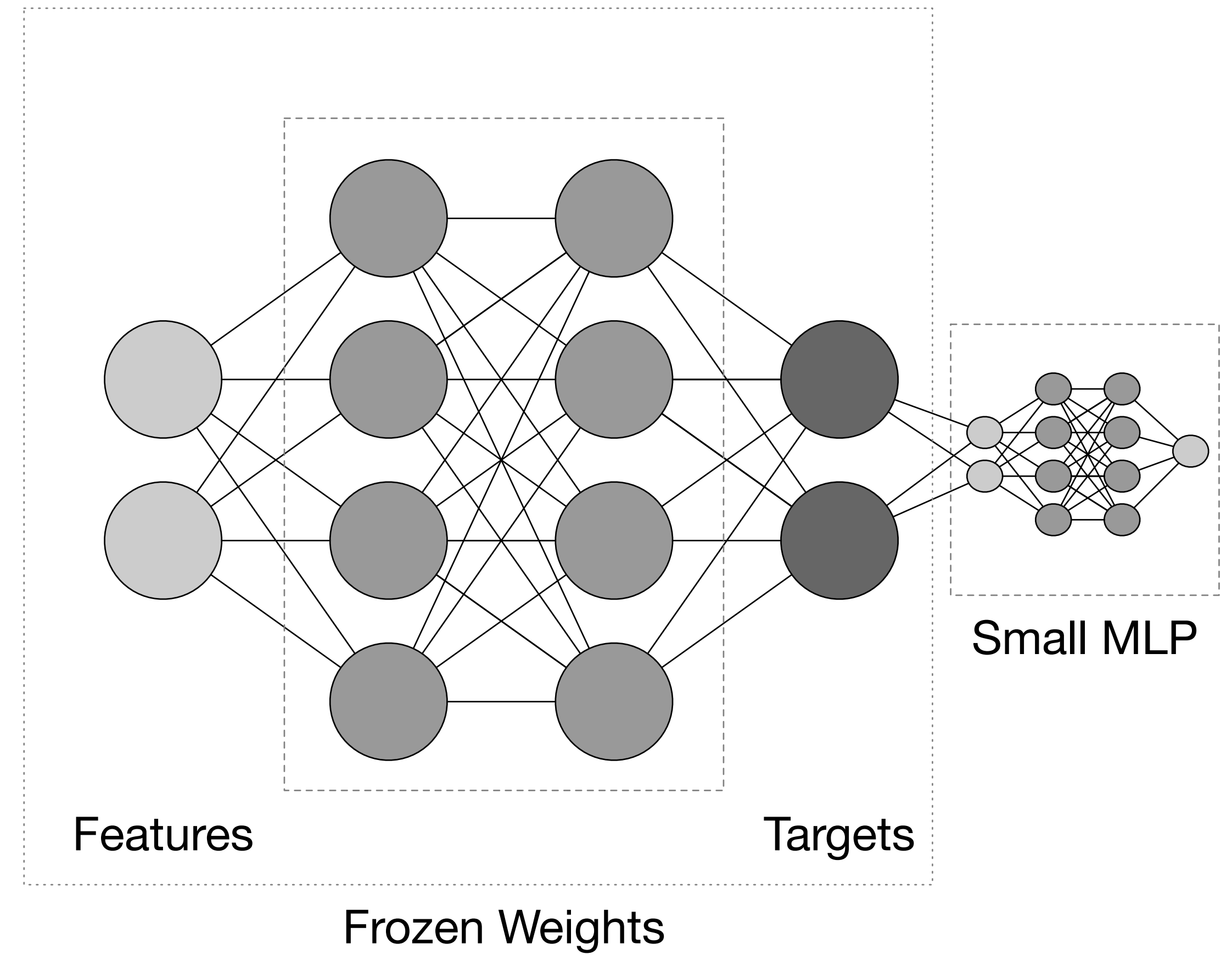
- Take the pre-trained multitask NN and train a linear model
- Weights of the Linear model are optimized by a gradient-free optimizer
- Tried a few different optimizers
- nevergrad is most flexible



Trick #2

Zeroth order optimizers

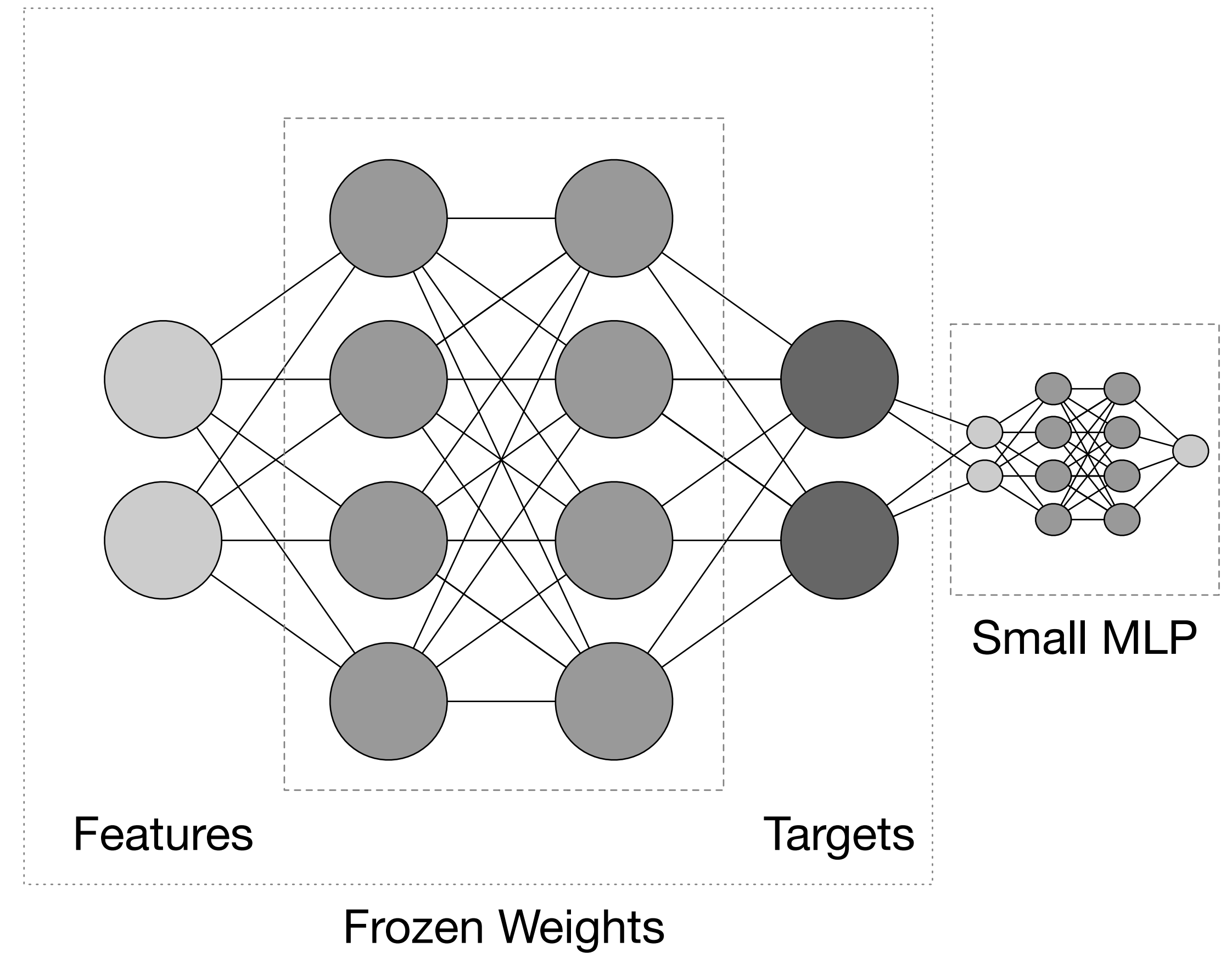
- Freeze the pre-trained NN



Trick #2

Zeroth order optimizers

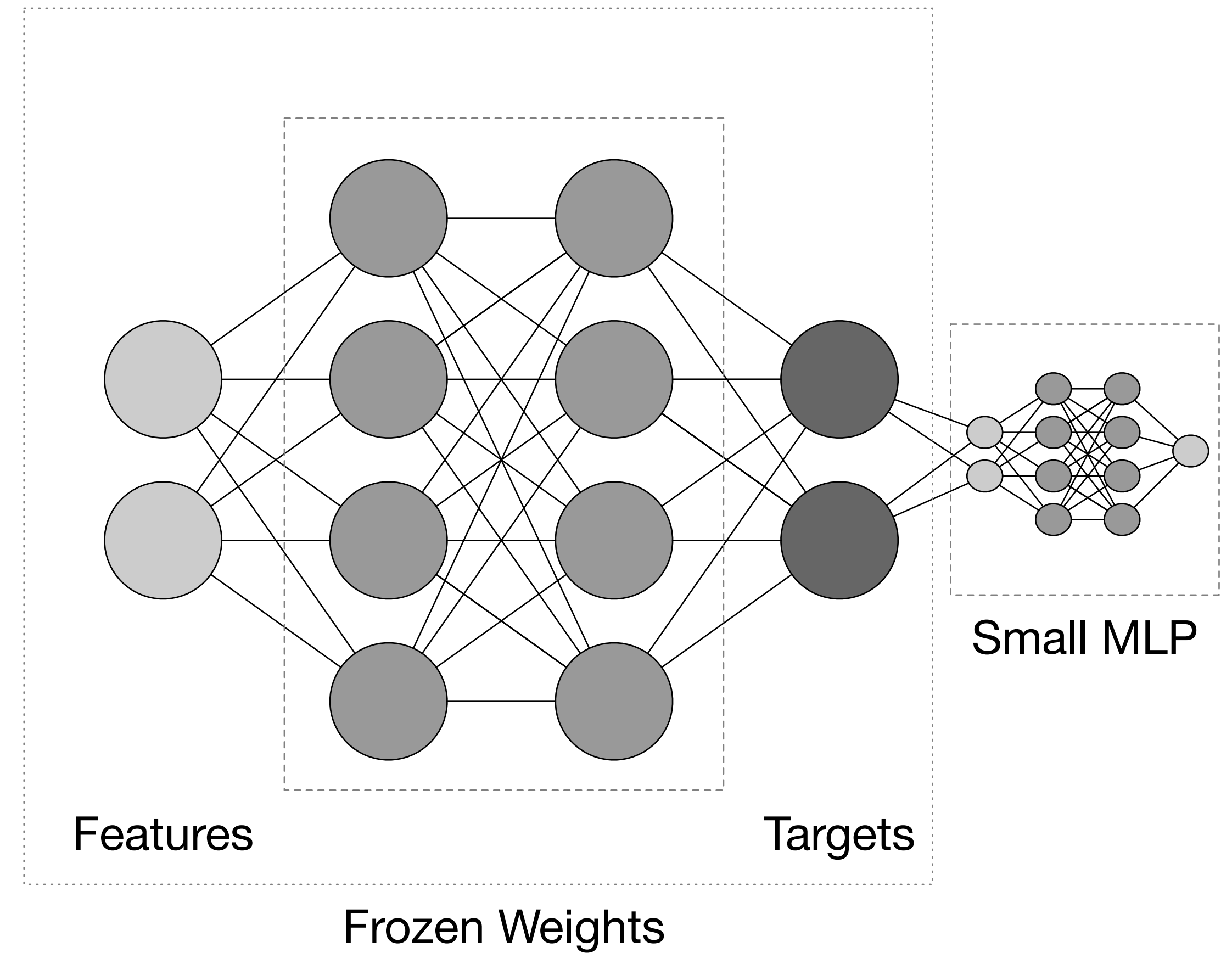
- Freeze the pre-trained NN
- Train a small NN on the outputs of the pre-trained NN with L2 loss



Trick #2

Zeroth order optimizers

- Freeze the pre-trained NN
- Train a small NN on the outputs of the pre-trained NN with L2 loss
- Then fine tune it on non-differentiable objective with a zero-th order optimizer.



Trick #2

Zeroth order optimizers (contd)

- Zeroth order optimizers is a whole subfield.

Algorithm 1: MeZO

Require: parameters $\theta \in \mathbb{R}^d$, loss $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$, step budget T , perturbation scale ϵ , batch size B , learning rate schedule $\{\eta_t\}$

```
for  $t = 1, \dots, T$  do
  Sample batch  $\mathcal{B} \subset \mathcal{D}$  and random seed  $s$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, \epsilon, s)$ 
   $\ell_+ \leftarrow \mathcal{L}(\theta; \mathcal{B})$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, -2\epsilon, s)$ 
   $\ell_- \leftarrow \mathcal{L}(\theta; \mathcal{B})$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, \epsilon, s)$  ▷ Reset parameters before descent

  projected_grad  $\leftarrow (\ell_+ - \ell_-)/(2\epsilon)$ 
  Reset random number generator with seed  $s$  ▷ For sampling  $z$ 
  for  $\theta_i \in \theta$  do
     $z \sim \mathcal{N}(0, 1)$ 
     $\theta_i \leftarrow \theta_i - \eta_t * \text{projected\_grad} * z$ 
  end
end

Subroutine  $\text{PerturbParameters}(\theta, \epsilon, s)$ 
  Reset random number generator with seed  $s$  ▷ For sampling  $z$ 
  for  $\theta_i \in \theta$  do
     $z \sim \mathcal{N}(0, 1)$ 
     $\theta_i \leftarrow \theta_i + \epsilon z$  ▷ Modify parameters in place
  end
  return  $\theta$ 
```

Trick #2

Zeroth order optimizers (contd)

- Zeroth order optimizers is a whole subfield.
- Projected gradients are noisy.

Algorithm 1: MeZO

Require: parameters $\theta \in \mathbb{R}^d$, loss $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$, step budget T , perturbation scale ϵ , batch size B , learning rate schedule $\{\eta_t\}$

```
for  $t = 1, \dots, T$  do
  Sample batch  $\mathcal{B} \subset \mathcal{D}$  and random seed  $s$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, \epsilon, s)$ 
   $\ell_+ \leftarrow \mathcal{L}(\theta; \mathcal{B})$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, -2\epsilon, s)$ 
   $\ell_- \leftarrow \mathcal{L}(\theta; \mathcal{B})$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, \epsilon, s)$  ▷ Reset parameters before descent

  projected_grad  $\leftarrow (\ell_+ - \ell_-)/(2\epsilon)$ 
  Reset random number generator with seed  $s$  ▷ For sampling  $z$ 
  for  $\theta_i \in \theta$  do
     $z \sim \mathcal{N}(0, 1)$ 
     $\theta_i \leftarrow \theta_i - \eta_t * \text{projected\_grad} * z$ 
  end
end
```

Subroutine $\text{PerturbParameters}(\theta, \epsilon, s)$

```
Reset random number generator with seed  $s$  ▷ For sampling  $z$ 
for  $\theta_i \in \theta$  do
   $z \sim \mathcal{N}(0, 1)$ 
   $\theta_i \leftarrow \theta_i + \epsilon z$  ▷ Modify parameters in place
end
return  $\theta$ 
```

Trick #2

Zeroth order optimizers (contd)

- Zeroth order optimizers is a whole subfield.
- Projected gradients are noisy.
- Slow convergence.

Algorithm 1: MeZO

Require: parameters $\theta \in \mathbb{R}^d$, loss $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$, step budget T , perturbation scale ϵ , batch size B , learning rate schedule $\{\eta_t\}$

```
for  $t = 1, \dots, T$  do
  Sample batch  $\mathcal{B} \subset \mathcal{D}$  and random seed  $s$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, \epsilon, s)$ 
   $\ell_+ \leftarrow \mathcal{L}(\theta; \mathcal{B})$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, -2\epsilon, s)$ 
   $\ell_- \leftarrow \mathcal{L}(\theta; \mathcal{B})$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, \epsilon, s)$  ▷ Reset parameters before descent

  projected_grad  $\leftarrow (\ell_+ - \ell_-)/(2\epsilon)$ 
  Reset random number generator with seed  $s$  ▷ For sampling  $z$ 
  for  $\theta_i \in \theta$  do
     $z \sim \mathcal{N}(0, 1)$ 
     $\theta_i \leftarrow \theta_i - \eta_t * \text{projected\_grad} * z$ 
  end
end
```

Subroutine $\text{PerturbParameters}(\theta, \epsilon, s)$

```
  Reset random number generator with seed  $s$  ▷ For sampling  $z$ 
  for  $\theta_i \in \theta$  do
     $z \sim \mathcal{N}(0, 1)$ 
     $\theta_i \leftarrow \theta_i + \epsilon z$  ▷ Modify parameters in place
  end
  return  $\theta$ 
```

Trick #2

Zeroth order optimizers (contd)

- Zeroth order optimizers is a whole subfield.
- Projected gradients are noisy.
- Slow convergence.
- But more efficient than evolutionary methods.

Algorithm 1: MeZO

Require: parameters $\theta \in \mathbb{R}^d$, loss $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$, step budget T , perturbation scale ϵ , batch size B , learning rate schedule $\{\eta_t\}$

```
for  $t = 1, \dots, T$  do
  Sample batch  $\mathcal{B} \subset \mathcal{D}$  and random seed  $s$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, \epsilon, s)$ 
   $\ell_+ \leftarrow \mathcal{L}(\theta; \mathcal{B})$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, -2\epsilon, s)$ 
   $\ell_- \leftarrow \mathcal{L}(\theta; \mathcal{B})$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, \epsilon, s)$  ▷ Reset parameters before descent

  projected_grad  $\leftarrow (\ell_+ - \ell_-)/(2\epsilon)$ 
  Reset random number generator with seed  $s$  ▷ For sampling  $z$ 
  for  $\theta_i \in \theta$  do
     $z \sim \mathcal{N}(0, 1)$ 
     $\theta_i \leftarrow \theta_i - \eta_t * \text{projected\_grad} * z$ 
  end
end

Subroutine  $\text{PerturbParameters}(\theta, \epsilon, s)$ 
  Reset random number generator with seed  $s$  ▷ For sampling  $z$ 
  for  $\theta_i \in \theta$  do
     $z \sim \mathcal{N}(0, 1)$ 
     $\theta_i \leftarrow \theta_i + \epsilon z$  ▷ Modify parameters in place
  end
return  $\theta$ 
```

Trick #2

Zeroth order optimizers (contd)

- Zeroth order optimizers is a whole subfield.
- Projected gradients are noisy.
- Slow convergence.
- But more efficient than evolutionary methods.
- Pretraining helps a lot.

Algorithm 1: MeZO

Require: parameters $\theta \in \mathbb{R}^d$, loss $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$, step budget T , perturbation scale ϵ , batch size B , learning rate schedule $\{\eta_t\}$

```
for  $t = 1, \dots, T$  do
  Sample batch  $\mathcal{B} \subset \mathcal{D}$  and random seed  $s$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, \epsilon, s)$ 
   $\ell_+ \leftarrow \mathcal{L}(\theta; \mathcal{B})$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, -2\epsilon, s)$ 
   $\ell_- \leftarrow \mathcal{L}(\theta; \mathcal{B})$ 
   $\theta \leftarrow \text{PerturbParameters}(\theta, \epsilon, s)$  ▷ Reset parameters before descent

  projected_grad  $\leftarrow (\ell_+ - \ell_-)/(2\epsilon)$ 
  Reset random number generator with seed  $s$  ▷ For sampling  $z$ 
  for  $\theta_i \in \theta$  do
     $z \sim \mathcal{N}(0, 1)$ 
     $\theta_i \leftarrow \theta_i - \eta_t * \text{projected\_grad} * z$ 
  end
end

Subroutine  $\text{PerturbParameters}(\theta, \epsilon, s)$ 
  Reset random number generator with seed  $s$  ▷ For sampling  $z$ 
  for  $\theta_i \in \theta$  do
     $z \sim \mathcal{N}(0, 1)$ 
     $\theta_i \leftarrow \theta_i + \epsilon z$  ▷ Modify parameters in place
  end
return  $\theta$ 
```

Trick #3

Score function estimator

- Numerai perhaps isn't a supervised machine learning problem anymore.

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}.$$

Trick #3

Score function estimator

- Numerai perhaps isn't a supervised machine learning problem anymore.
- The problem is to pick predictor to maximize the payout, subject to certain constraints.

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}.$$

Trick #3

Score function estimator

- Numerai perhaps isn't a supervised machine learning problem anymore.
- The problem is to pick predictor to maximize the payout, subject to certain constraints.
- REINFORCE is a gradient estimator, that also happens to be a policy gradient method in RL.

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}.$$

Trick #3

Score function estimator

- Numerai perhaps isn't a supervised machine learning problem anymore.
- The problem is to pick predictor to maximize the payout, subject to certain constraints.
- REINFORCE is a gradient estimator, that also happens to be a policy gradient method in RL.
- Still WIP.

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla \pi(A_t | S_t, \boldsymbol{\theta}_t)}{\pi(A_t | S_t, \boldsymbol{\theta}_t)}.$$

Fin